

3.1. Representing Knowledge with Real Logic

In logic-based knowledge representation systems, knowledge is represented by logical formulas whose intended meanings are propositions about a domain of interest. The connection between the symbols occurring in the formulas and what holds in the domain is not represented in the knowledge base and is left implicit since it does not have any effect on the logic computations. In Real Logic, by contrast, the connection between the symbols and the domain is represented explicitly in the language by the grounding $\mathcal{G}$, which plays an important role in both learning and reasoning. $\mathcal{G}$ is an integral part of the knowledge represented by Real Logic. A Real Logic knowledge base is therefore defined by the formulas of the logical language and knowledge about the domain in the form of groundings obtained from data. The following types of knowledge can be represented in Real Logic.

3.1.1. Knowledge through symbol groundings

Boundaries for domain grounding. These are constraints specifying that the value of a certain logical expression must be within a certain range. For instance, one may specify that the domain D must be interpreted in the $[0, 1]$ hyper-cube or in the standard n -simplex, i.e. the set $d_1, \dots, d_n \in (\mathbb{R}^+)^n$ such that $\sum_i d_i = 1$. Other intuitive examples of range constraints include the elements of the domain “colour” grounded onto points in $[0, 1]^3$ such that every element is associated with the triplet of values (R, G, B) with $R, G, B \in [0, 1]$, or the range of a function $age(x)$ as an integer between 0 and 100.

Explicit definition of grounding for symbols. Knowledge can be more strictly incorporated by fixing the grounding of some symbols. If a constant c denotes an object with known features $\mathbf{v}_c \in \mathbb{R}^n$, we can fix its grounding $\mathcal{G}(c) = \mathbf{v}_c$. Training data that consists in a set of n data items such as n images (or tuples known as training examples) can be specified in Real Logic by n constants, e.g. $img_1, img_2, \dots, img_n$, and by their groundings, e.g. $\mathcal{G}(img_1) = \begin{bmatrix} 0.1 \\ 0.2 \\ 0.3 \end{bmatrix}$, $\mathcal{G}(img_2) = \begin{bmatrix} 0.4 \\ 0.5 \\ 0.6 \end{bmatrix}$, $\dots$, $\mathcal{G}(img_n) = \begin{bmatrix} 0.7 \\ 0.8 \\ 0.9 \end{bmatrix}$. These can be gathered in a variable $imgs$. A binary predicate sim that measures the similarity of two objects can be grounded as, e.g., a cosine similarity function of two vectors $\mathbf{v}$ and $\mathbf{w}$, $(\mathbf{v}, \mathbf{w}) \mapsto \frac{\mathbf{v} \cdot \mathbf{w}}{\|\mathbf{v}\| \|\mathbf{w}\|}$. The output layer of the neural network associated with a multi-class single-label predicate $P(x, class)$ can be a `softmax` function normalizing the output such that it guarantees exclusive classification, i.e. $\sum_i P(x, i) = 1$.⁹ Grounding of constants and functions allows the computation of the grounding of their results. If, for example, $\mathcal{G}(transp)$ is the function that transposes a matrix then $\mathcal{G}(transp(img_1)) = \begin{bmatrix} 0.1 & 0.4 & 0.7 \end{bmatrix}$.

Parametric definition of grounding for symbols. Here, the exact grounding of a symbol σ is not known, but it is known that it can be obtained by finding a set of real-valued parameters, that is, via learning. To emphasize this fact, we adopt the notation $\mathcal{G}(\sigma) = \mathcal{G}(\sigma \mid \theta_\sigma)$ where θ_σ is the set of parameter values that determines the value of $\mathcal{G}(\sigma)$. The typical example of parametric grounding for constants is the learning of an embedding. Let $emb(word \mid \theta_{emb})$ be a word embedding with parameters θ_{emb} which takes as input a word and returns its embedding in $\mathbb{R}^n$. If the words of a vocabulary $W = \{w_1, \dots, w_{|W|}\}$ are constant symbols,

⁹Notice that `softmax` is often used as the last layer in neural networks to turn logits into a probability distribution. However, we do not use the `softmax` function as such here. Instead, we use it here to enforce an exclusivity constraint on satisfiability scores.

their groundings $\mathcal{G}(w_i \mid \theta_{emb})$ are defined parametrically w.r.t. θ_{emb} as $emb(w_i \mid \theta_{emb})$. An example of parametric grounding for a function symbol f is to assume that $\mathcal{G}(f)$ is a linear function such that $\mathcal{G}(f) : \mathbb{R}^m \rightarrow \mathbb{R}^n$ maps each $\mathbf{v} \in \mathbb{R}^m$ into $\mathbf{A}_f \mathbf{v} + \mathbf{b}_f$, with $\mathbf{A}_f$ a matrix of real numbers and $\mathbf{b}$ a vector of real numbers. In this case, $\mathcal{G}(f) = \mathcal{G}(f \mid \theta_f)$, where $\theta_f = \{\mathbf{A}_f, \mathbf{b}_f\}$. Finally, the grounding of a predicate symbol can be given, for example, by a neural network N with parameters θ_N . As an example, consider a neural network N trained for image classification into n classes: *cat*, *dog*, *horse*, etc. N takes as input a vector $\mathbf{v}$ of pixel values and produces as output a vector $\mathbf{y} = (y_{cat}, y_{dog}, y_{horse}, \dots)$ in $[0, 1]^n$ such that $\mathbf{y} = N(\mathbf{v} \mid \theta_N)$, where y_c is the probability that input image $\mathbf{v}$ is of class c . In case classes are, alternatively, chosen to be represented by unary predicate symbols such as *cat*($\mathbf{v}$), *dog*($\mathbf{v}$), *horse*($\mathbf{v}$), ... then $\mathcal{G}(\text{cat}(\mathbf{v})) = N(\mathbf{v} \mid \theta_N)_{cat}$, $\mathcal{G}(\text{dog}(\mathbf{v})) = N(\mathbf{v} \mid \theta_N)_{dog}$, $\mathcal{G}(\text{horse}(\mathbf{v})) = N(\mathbf{v} \mid \theta_N)_{horse}$, etc.

3.1.2. Knowledge through formulas

Factual propositions. Knowledge about the properties of specific objects in the domain is represented, as usual, by logical propositions, as exemplified below: Suppose that it is known that img_1 is a number eight, img_2 is a number nine, and img_n is a number two. This can be represented by adding the following facts to the knowledge-base: $nine(img_1)$, $eight(img_2)$, ..., $two(img_n)$. Supervised learning, that is, learning with the use of training examples which include target values (labelled data), is specified in Real Logic by combining grounding definitions and factual propositions. For example, the fact that an image img_1 is a positive example for the class nine and a negative example for the class eight is specified by defining $\mathcal{G}(img_1) = \text{img}_1$ alongside the propositions $nine(img_1)$ and $\neg eight(img_1)$. Notice how semi-supervision can be specified naturally in Real Logic by adding propositions containing disjunctions, e.g. $eight(img_1) \vee nine(img_1)$, which state that img_1 is either an eight or a nine (or both). Finally, relational learning can be achieved by relating logically multiple objects (defined as constants or variables or even as more complex sequences of terms) such as e.g.: $nine(img_1) \rightarrow \neg nine(img_2)$ (if img_1 is a nine then img_2 is not a nine) or $nine(img) \rightarrow \neg eight(img)$ (if an image is a nine then it is not an eight). The use of more complex knowledge including the use of variables such as img above is the topic of *generalized propositions*, discussed next.

Generalized propositions. General knowledge about all or some of the objects of some domains can be specified in Real Logic by using first-order logic formulas with quantified variables. This general type of knowledge allows one to specify arbitrary constraints on the groundings independently from the specific data available. It allows one to specify, in a concise way, knowledge that holds true for all the objects of a domain. This is especially useful in Machine Learning in the semi-supervised and unsupervised settings, where there is no specific knowledge about a single individual. For example, as part of a task of multi-label classification with constraints on the labels [12], a positive label constraint may express that if an example is labelled with $l_1, \dots, l_k$ then it should also be labelled with l_{k+1} . This can be specified in Real Logic with a universally quantified formula: $\forall x (l_1(x) \wedge \dots \wedge l_k(x) \rightarrow l_{k+1}(x))$.¹⁰ Another example of soft constraints used in Statistical Relational Learning associates the labels of related examples. For instance, in Markov Logic Networks [55], as part of the well-known

¹⁰This can also be specified using a guarded quantifier $\forall x : ((l_1(x) \wedge \dots \wedge l_k(x)) > th) \rightarrow l_{k+1}(x)$ where th is a threshold value in $[0, 1]$.

Smokers and Friends example, people who are smokers are associated by the friendship relation. In Real Logic, the formula $\forall xy ((\text{smokes}(x) \wedge \text{friend}(x, y)) \rightarrow \text{smokes}(y))$ would be used to encode the soft constraint that friends of smokers are normally smokers.

3.1.3. Knowledge through fuzzy semantics

Definition for operators. The grounding of a formula ϕ depends on the operators approximating the connectives and quantifiers that appear in ϕ . Different operators give different interpretations of the satisfaction associated with the formula. For instance, the operator $A_{pME}(a_1, \dots, a_n)$ that approximates universal quantification can be understood as a smooth minimum. It depends on a hyper-parameter p (the exponent used in the generalized mean). If $p = 1$ then $A_{pME}(a_1, \dots, a_n)$ corresponds to the arithmetic mean. As p increases, given the same input, the value of the universally quantified formula will decrease as A_{pME} converges to the min operator. To define how strictly the universal quantification should be interpreted in each proposition, one can use different values of p for different propositions of the knowledge base. For instance, a formula $\forall x P(x)$ where A_{pME} is used with a low value for p will in fact denote that P holds for *some* x , whereas a formula $\forall x Q(x)$ with a higher p may denote that Q holds for *most* x .

3.1.4. Satisfiability

In summary, a Real Logic knowledge-base has three components: the first describes knowledge about the grounding of symbols (domains, constants, variables, functions, and predicate symbols); the second is a set of closed logical formulas describing factual propositions and general knowledge; the third lies in the operators and the hyperparameters used to evaluate each formula. The definition that follows formalizes this notion.

Definition 4 (Theory/Knowledge-base). A *theory* of Real Logic is a triple $\mathcal{T} = \langle \mathcal{K}, \mathcal{G}(\cdot | \theta), \Theta \rangle$, where $\mathcal{K}$ is a set of closed first-order logic formulas defined on the set of symbols $S = D \cup X \cup C \cup F \cup P$ denoting, respectively, domains, variables, constants, function and predicate symbols; $\mathcal{G}(\cdot | \theta)$ is a parametric grounding for all the symbols $s \in S$ and all the logical operators; and $\Theta = \{\Theta_s\}_{s \in S}$ is the hypothesis space for each set of parameters θ_s associated with symbol s .

Learning and reasoning in a Real Logic theory are both associated with searching and applying the set of values of parameters θ from the hypothesis space Θ that maximize the satisfaction of the formulas in $\mathcal{K}$. We use the term *grounded theory*, denoted by $\langle \mathcal{K}, \mathcal{G}_\theta \rangle$, to refer to a Real Logic theory with a specific set of learned parameter values. This idea shares some similarity with the weighted MAX-SAT problem [43], where the weights for formulas in $\mathcal{K}$ are given by their fuzzy truth-values obtained by choosing the parameter values of the grounding. To define this optimization problem, we aggregate the truth-values of all the formulas in $\mathcal{K}$ by selecting a *formula aggregating* operator $\text{SatAgg} : [0, 1]^* \rightarrow [0, 1]$.

Definition 5. The *satisfiability* of a theory $\mathcal{T} = \langle \mathcal{K}, \mathcal{G}_\theta \rangle$ with respect to the *aggregating operator* SatAgg is defined as $\text{SatAgg}_{\phi \in \mathcal{K}} \mathcal{G}_\theta(\phi)$.

3.2. Learning

Given a Real Logic theory $\mathcal{T} = (\mathcal{K}, \mathcal{G}(\cdot | \theta), \Theta)$, *learning* is the process of searching for the set of parameter values θ^* that maximize the satisfiability of $\mathcal{T}$ w.r.t. a given aggregator:

$$\theta^* = \underset{\theta \in \Theta}{\operatorname{argmax}} \underset{\phi \in \mathcal{K}}{\text{SatAgg}} \mathcal{G}_\theta(\phi)$$